

HOMework 4

CMU 10-703: DEEP REINFORCEMENT LEARNING (FALL 2026)

OUT: Wednesday, September 30, 2026

DUE: Friday, October 9, 2026 by 11:59pm ET

Instructions: START HERE

Note: This homework assignment requires a significant implementation effort. Please plan your time accordingly. General tips and suggestions are included in the ‘Guidelines on Implementation’ section at the end of this handout.

- **Collaboration policy:** You may work in groups of up to three people for this assignment. It is also OK to get clarification (but not solutions) from books or online resources after you have thought about the problems on your own. You are expected to comply with the University Policy on Academic Integrity and Plagiarism¹.
- **Late Submission Policy:** You are allowed a total of 10 grace days for your homeworks. **For this assignment only, if you use a late day, you will be extended the entire fall break (due date would be Sunday, October 18). Note that you will at most use a single day for this HW.** Grace days do not need to be requested or mentioned in emails; we will automatically apply them to students who submit late. We will not give any further extensions so make sure you only use them when you are absolutely sure you need them. See the Assignments and Grading Policy here for more information about grace days and late submissions: https://cmudeeprl.github.io/703website_f26/
- **Submitting your work:**
 - **Gradescope:** Please write your answers and copy your plots into the provided LaTeX template, and upload a PDF to the GradeScope assignment titled “Homework 4.” Additionally, zip all the code folders into a directory titled `<andrew_id>.zip` and upload it the GradeScope assignment titled “Homework 4: Code.” Each team should only upload one copy of each part. Regrade requests can be made within one week of the assignment being graded.
 - **Submitting GIFs:** For each GIF, upload the file to Google Drive, set its sharing permission to “Anyone with the link” (Viewer), and paste the shareable link into the matching box of the submission template. Use the file name given in each question (`gifs_BC.gif`, `gifs_DAgger.gif`, `gifs_diffusion.gif`). Make sure the link is viewable without requiring the graders to request access.

¹<https://www.cmu.edu/policies/>

Problem 0: Collaborators

Please list your name and Andrew ID, as well as those of your collaborators.

Problem 1: CMA-ES (24 pts)

In this problem you will implement CMA-ES, a black-box optimization algorithm. To help you get started, we have provided some template code in this Notebook:

https://colab.research.google.com/drive/1jjRDAQ3l78CZTZ_FKZKLaABlb0rFgYsg?usp=sharing

You are welcome to implement the assignment from scratch using whatever programming language you prefer.

1. [10 pts] Implement CMA-ES using the following update equations:

$$\mu_{t+1} \leftarrow \frac{1}{\text{elite size}} \sum_{i=1}^{\text{elite size}} \theta_t^{(i)}, \quad \Sigma_t \leftarrow \text{Cov} \left(\theta_t^{(1)}, \dots, \theta_t^{(\text{elite size})} \right) + \epsilon I,$$

where $\theta_t^{(i)}$ denotes the i -th best parameters from the previous iteration and $\epsilon \in \mathbb{R}$ is a small constant. We recommend the following hyperparameters:

- Initial μ : $\vec{0}$
- Initial covariance: $100I$
- Population size: 100
- Fraction of population to keep at each iteration: 10%
- Noise ϵ added to covariance at each step: $0.25I$

Run your implementation of CMA-ES to *maximize* on the following simple objective function:

$$f(x) = -\|x - x^*\|_2^2 \quad \text{where} \quad x^* = [65, 49].$$

This function is optimized when $x = x^*$. Run your implementation of CMA-ES on this function, confirming that you get the correct solution. Make a plot showing the values of μ from each iteration. Use $\mu[0]$ for the X axis and $\mu[1]$ for the Y axis. Please label the initial and final values of μ , as well as the global optimum. Remember to label your axes.

2. [4 pts] In the second part of this problem, you will use CMA-ES to solve a RL problem. You will use the `CartPole-v0` environment from Gymnasium. Our first task will be to make an objective function that takes as input the parameters of a policy and outputs the reward of that policy. We will parametrize the policy as

$$\pi(a = \text{LEFT} \mid s) = \sigma(s \cdot w + b),$$

where σ is the sigmoid function, and $w \in \mathbb{R}^4$ and $b \in \mathbb{R}$ are parameters that you will optimize with CMA-ES. Define a function that takes as input a single vector

$x = (w, b)$ and the environment and returns the total (undiscounted) reward from one episode. End the episode when `env.step()` returns either `terminated` or `truncated` as true (`CartPole-v0` truncates episodes at 200 steps). To check your implementation, evaluate the following policies 1000 times and report the average total reward (we've provided the answer for the first policy). The 1000-episode average is only for this check; in part 3, CMA-ES scores each sample with a single call to this function (one episode).

$x = (-1, -1, -1, -1, -1)$	$x = (1, 0, 1, 0, 1)$	$x = (0, 1, 2, 3, 4)$
15.6		

3. [10 pts] Run CMA-ES on the RL objective function. CMA-ES should be able to get an average population reward of at least 195 in 10 iterations. Include plot showing mean sample reward and best sample reward (Y axis) across iterations (X axis). Remember to label both lines and both axes.

Problem 2: Imitation Learning (62 pts)

In this problem, you will implement imitation learning from demonstrations. We will be using the `BipedalWalker-v3` environment. The expert trajectories to imitate will be supplied by an algorithm, specifically the RL method of Proximal Policy Optimization (PPO) [8], instead of human experts. You will have access to both the trajectories for BC and the actual PPO policy for DAgger. The expert trajectories to use are `actions_BC.pkl` and `states_BC.pkl` and are stored at *this link*. The checkpoint for the expert policy is at `data/models/super_expert_PPO_model.pt`. Use the provided `environment.yaml` for compatibility.

2.1. Behaviour Cloning (14 pts)

First you will implement Behavior Cloning. In behavior cloning, we simply use regression to train a policy to map observations to actions, supervised by a set of expert trajectories, i.e., sequences of state and actions. For the sake of evaluation, we will also be monitoring the reward of the training and generated trajectory segments. Our policy architecture is a residual three layer multilayer perceptron, and outputs actions in the range $[-1, 1]$.

The reference solutions takes less than 5 minutes to train and generate visualizations for each (with a GPU).

1. Implement the `generate_trajectories`, `train`, and `run_training` functions:
 - Implement the `generate_trajectories` function, which simply runs inference for a given number of trajectories with `self.model` using the `generate_trajectory` method and returns the rewards.
 - Implement the `train` function to run training when `self.mode` is "BC". We have provided a function that computes gradients and updates the model already, called `training_step`. In `train`, you should loop for `num_batch_collection_steps` steps. For each batch collection step, perform `num_training_steps_per_batch`

training steps using `training_step`. Note that since we provide the expert trajectories, you don't actually need to rollout the expert policy to get these trajectories, just sample them from the data. For evaluation, compute/store the average, median, and max reward from the `num_trajectories_per_batch` trajectories every `num_training_steps_per_batch` training steps using the function implemented previously.

- For `run_training`, create a SimpleNet model with the following parameters:
 - `hidden_layer_dimension`: 128
 - `max_episode_length`: 1600

Use the AdamW optimizer with the following parameters:

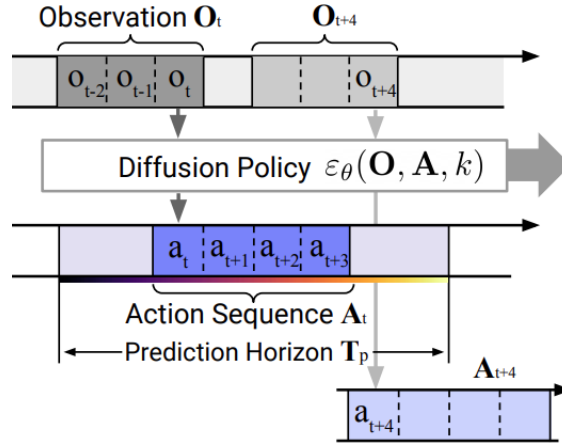
- learning rate: 0.0001
- weight decay: 0.0001
- Pass all necessary information into a `TrainDaggerBC` object (with `mode = "BC"`), which can then be trained with the `TrainDaggerBC.train()` method. Collect 20 batches, each with 20 trajectories per batch. Train for 1000 steps per batch with a batch size of 128.

2. [6 pts] Save the training loss plot, *and write down the final loss value*.
3. [6 pts] Create a graph of the average, median, and max reward for trajectories sampled with your trained policy. Report these statistics for 20 trajectories every 1000 training steps until training is concluded.
4. [2 pts] After training, create and save a gif of the walker during a failure run of behavior cloning (reward below zero). Save the .gif as "`gifs_BC.gif`", upload it to Google Drive, share it with view access, and paste the link in the matching box of the submission template.

2.2. DAgger (18 pts)

Next, you will implement the DAgger algorithm [7] in `train_dagger_BC.py`. We have provided pseudocode in Algorithms 1 and 2.

1. Implement the `update_training_data` function, and update the `run_training` and `train` functions:
 - `update_training_data` should follow Algorithm 2.
 - Update the `train()` function to follow Algorithm 1 when `self.mode` is DAgger where instead of simply generating trajectory using our model, we also query the expert policy.
 - Use the same hyperparameters as the BC with the same initial model but set `mode = "DAgger"`. Collect 20 batches, each with 20 trajectories per batch. Train for 1000 steps per batch with a batch size of 128.



a) Diffusion Policy General Formulation

(a) Inputs and outputs of the diffusion policy [2]. We take in previous observations (previous states and actions in our case), and output a set of future actions (3 actions in our case)

Figure 1

2. **[6 pts]** Save the training loss plot (it may have some spikes), *and write down the final loss value*.
3. **[6 pts]** Create a graph of the average, median, and max reward for each `batch_collection_step` when collecting trajectories using the DAgger model, and plot the results.
4. **[2 pts]** After training, create and save a gif of the walker during a successful run of DAgger (reward above 260). Save the .gif as "`gifs_DAgger.gif`", upload it to Google Drive, share it with view access, and paste the link in the matching box of the submission template.
5. **[4 pts]** Which method worked better, BC or DAgger, and why?

2.3. Diffusion policies [2, 3] (30 pts)

Next, you will implement learning from demonstrations with diffusion policies. Our policy architecture is transformer that takes as input a history of states and actions with their corresponding episode timesteps, noisy future actions and predicts the noise. The policy predicts a short sequence of future actions rather than a single action; Section 4.3 of [2] explains why.

We provide a high level overview in Figure 1 as well as the architecture itself in `diffusion_policy_transformer.py`. If you have never seen a `torch.nn.module` implementation of a transformer network with cross attention/diffusion model, it may be interesting to look at `diffusion_policy_transformer.py`. Your implementation will be in `train_diffusion_policy.py`.

The reference solution takes around 30 minutes to train and generate visualizations (with a GPU).

1. Implement the diffusion policy by updating the `run_training`, `training_step`, `diffusion_sample` and `sample_trajectory` methods of `train_diffusion_policy.py`.
 - (a) Implement the `training_step` function in `train_diffusion_policy.py` via Algorithm 3. We will use Denoising Diffusion Probabilistic Models [4] with 30 denoising timesteps as our diffusion implementation, and this algorithm is a rough version of Algorithm 1 from the paper.
 - (b) For `run_training`, create a `PolicyDiffusionTransformer` model with the following parameters:
 - 6 transformer layers
 - hidden size 128
 - 1 transformer head

Use the AdamW optimizer with the following parameters:

- learning rate: 0.00005
- weight decay: 0.001

Load the states and actions (`states_BC.pkl` and `actions_BC.pkl`, from the same link as in the introduction to Problem 2) using `pickle.load`, and pass all necessary information into a `TrainDiffusionPolicy` object, which can then be trained with the `TrainDiffusionPolicy.train()` method. Train for 50000 steps with a batch size of 256. Save a model after 50000 steps.

(note) If your loss is below 0.01 or above 0.15 after 10000 iterations, your training is probably buggy - it is recommended to use wandb for viewing loss curves.

- (c) [6 pts] Submit a training loss curve, and *write down the final loss value*
- (d) Implement the `diffusion_sample` function as in algorithm 4
- (e) Implement the `sample_trajectory` function using the diffusion transformer, as seen in Algorithm 5. Here are some helpful tips:
 - During training, we normalized state values using `self.states_mean` and `self.states_std` before passing them into the model and normalized action values using `self.actions_mean` and `self.actions_std` before having the model predict them (this happened during initialization of `TrainDiffusionPolicy`). Make sure to account for this when running inference.
 - During the beginning of inference, you will not have enough previous states/actions for full conditioning. Make sure to pad accordingly - padding should be False for states/actions to be included in the sequence, and True for states/actions to be padded, and padded states should be added to the end of the previous actions/previous states tensors (i.e. higher indices along the `input_seq_length` dimension, see the `nn.module` for `diffusion_policy_transformer`).

(**note**) for the following parts, feel free to update the **evaluation** method

- (f) [**15 pts**] Generate 20 trajectories with `num_actions_to_eval_in_a_row=1`, 2, and 3, and calculate average time to run each process (This will take a long time to run. It is recommended to start with 3 actions evaluated in a row to test your setup; with 3 actions in a row, the max reward should be above 240). What is the max, median, and mean reward in each case, and what is the average time to generate a trajectory in each case? Expect a large spread in both reward and time across the 20 trajectories; this is normal.
- (g) [**2 pts**] After training, create and save a gif of the walker during a successful run of the diffusion policy (reward above 240) with 3 actions evaluated in a row. Save the .gif as "**gifs_diffusion.gif**", upload it to Google Drive, share it with view access, and paste the link in the matching box of the submission template.
- (h) [**4 pts**] Why does the diffusion policy takes so much longer to generate a trajectory than the simple model trained using DAgger or BC?
- (i) [**3 pts**] Explain the difference in average time between your different test runs in part (f).

Feedback

Feedback: You can help the course staff improve the course by providing feedback. What was the most confusing part of this homework, and what would have made it less confusing?

Time Spent: How many hours did you spend working on this assignment? Your answer will not affect your grade.

Algorithms

The algorithms referenced throughout Problem 2 are collected here for reference.

Algorithm 1 DAgger

Policy π_θ , Expert policy π_ϕ^* , num_batch_collection_steps= n , num_steps_per_batch= m

```
1: procedure TRAIN
2:   for  $i$  in range( $n$ ):
3:     query_expert_policy( $\pi_\theta, \pi_\phi^*$ )
4:     for  $j$  in range( $m$ ):
5:       # gets a minibatch and does a gradient update of  $\pi_\theta$ 
6:       training_step( $\pi_\theta, \mathcal{D}$ )
7: end procedure
```

Algorithm 2 Query expert policy

Policy π_θ , Expert policy π^* , num_trajectories= N , current dataset $\mathcal{D}$

```
1: procedure QUERY_EXPERT_POLICY
2:   for  $i = 1$  to  $N$  do
3:     Sample trajectory  $\tau_i = [s_i^1, a_i^1, \dots, s_i^T, a_i^T]$  by deploying policy  $\pi_\theta$ .
4:     Get expert labels on the states of  $\tau_i$  :  $\mathcal{D}_i = \{(s_i^t, \pi^*(s_i^t), t = 1..T)\}$ .
5:     Aggregate datasets:  $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_i$ .
6: end procedure
```

Algorithm 3 Diffusion Policy Training Step

Denoising model ϵ_θ **NOTE:** padding + episode timesteps removed for brevity

```
1: procedure TRAINING_STEP(batch_size)
2:   # our model conditions on  $k$  input states,  $k - 1$  input actions,
3:   # and denoises  $k'$  future actions
4:    $s_{i-k+1}, \dots, s_i, a_{i-k+1}, \dots, a_{i-1}, a_i, \dots, a_{i+k'-1} = \text{get\_training\_batch}(\text{batch\_size})$ 
5:    $\epsilon \sim \mathcal{N}(0, I)$  # noise should be i.i.d across all actions/dimensions
6:   # sample one  $\epsilon$  per batch element
7:    $t \sim \text{RandInt}([1, T])$  # sample one  $t$  per batch element (i.i.d)
8:   # use  $T = \text{self.num\_train\_diffusion\_timesteps}$ 
9:   # in code (0-indexed):  $t = \text{torch.randint}(0, T, (\text{batch\_size},))$ 
10:  #  $(a_i)_t$  is  $t$  noised action  $a_i$ 
11:   $[(a_i)_t, \dots, (a_{i+k'-1})_t] = \sqrt{\bar{\alpha}_t} * [a_i, \dots, a_{i+k'-1}] + \sqrt{1 - \bar{\alpha}_t} * \epsilon$ 
12:  # prev line implemented as:
13:  #  $\text{noisy\_actions} = \text{self.training\_scheduler.add\_noise}(\text{actions}, \epsilon, t)$ 
14:   $L(\theta) = \frac{1}{\text{batch\_size}} \|\epsilon_\theta(s_{i-k+1}, \dots, s_i, a_{i-k+1}, \dots, a_{i-1}, (a_i)_t, \dots, (a_{i+k'-1})_t, t) - \epsilon\|_2^2$ 
15:  # prev line implemented in torch as:  $\text{loss} = \text{torch.nn.MSELoss}()(\epsilon_\theta(\cdot), \epsilon)$ 
16:  Update weights  $\theta$  with: AdamW( $\nabla_\theta L(\theta)$ )
17: end procedure
```

Algorithm 4 Sampling from Diffusion Policy

Denoising model ϵ_θ **NOTE:** padding + episode timesteps removed for brevity

```
1: procedure DIFFUSION_SAMPLE(prev_states, prev_actions)
2:    $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  # pure noise that will become actions
3:    $\mathbf{x}_t = \mathbf{x}_T$ 
4:    $\text{timesteps} = \text{get\_inference\_timesteps}()$  # these will be in decreasing order  $T, \dots, 1$ 
5:   # (in code they are numbered from 0, i.e.  $T - 1, \dots, 0$ )
6:   for  $t$  in  $\text{timesteps}$ :
7:      $\text{noise\_levels} = t$ 
8:      $\text{model\_inputs} = \text{prev\_states}, \text{prev\_actions}, \text{noisy\_actions}, \text{noise\_levels}$ 
9:      $z \sim \mathcal{N}(0, I)$  if  $t > 1$  else  $z = 0$ 
10:     $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(\text{model\_inputs}) \right) + \sigma_t \mathbf{z}$  # here we slightly denoise actions
11:    # prev 2 lines implemented as:
12:    #  $\mathbf{x}_t = \text{self.inference\_scheduler.step}(\epsilon_\theta(\text{model\_inputs}), \text{noise\_levels}, \mathbf{x}_t)$ 
13:  Return  $\mathbf{x}_0$ 
14: end procedure
```

Algorithm 5 Sampling TrajectoryDenoising model ϵ_θ **NOTE:** padding removed for brevity

```
1: procedure SAMPLE_TRAJECTORY(num_prev_states= $K$ , num_actions_to_eval_in_a_row= $N$ )
2:    $s, a, t, \text{done}, \text{truncated} = [\text{env.reset()}], [], [0], \text{False}, \text{False}$ 
3:   while not done and not truncated:
4:     # sample next actions
5:     actions = diffusion_sample( $\epsilon_\theta, s, a, t$ )
6:     for  $i$  in range( $N$ ):
7:       new_s, r, done, truncated, _ = env.step(actions[i])
8:       if done or truncated:
9:         return
10:    # append new tokens to sequence
11:     $s, a, t = s + [\text{new\_s}], a + [\text{actions}[i]], t + [t[-1] + 1]$ 
12:    # context length of transformer is at most  $K$ 
13:     $s, a, t = s[-K:], a[-(K-1):], t[-K:]$ 
14: end procedure
```

Additional Questions [Not Graded]

1. Forward Diffusion

Consider the forward diffusion equation

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \epsilon_t \quad (1)$$

for $\epsilon_t \sim \mathcal{N}(0, \mathbf{I})$, $\epsilon_t \perp \mathbf{x}_{t-1}$. Assuming that $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$, it is easy to show for all t that $\mathbf{x}_t$ also has mean 0. Show that the covariance is also equal, i.e. $\mathbf{E}[\mathbf{x}_t^\top \mathbf{x}_t] = \mathbf{E}[\mathbf{x}_{t-1}^\top \mathbf{x}_{t-1}]$, and explain why this is significant.

2. Cosine Scheduling in Diffusion

Nicole and Dhariwal [5] introduced the cosine schedule for diffusion models:

$$\bar{\alpha}_t = \frac{\cos^2\left(\frac{t/T + s}{s+1} \frac{\pi}{2}\right)}{\cos^2\left(\frac{s}{s+1} \frac{\pi}{2}\right)}, \quad \beta_t = \min\left(1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}, \beta_{\max}\right), \quad (2)$$

where T is the total number of steps, $\beta_{\max}$ is a cap on the β_t values ($\beta_{\max} = 0.999$ in the original paper), and s is some small offset ($s = 0.008$ in the original paper) to ensure that β_t remains non-zero. Compared to a linear scheduler given by $\beta_t = \beta_{\min} + \frac{t-1}{T-1}(\beta_{\max} - \beta_{\min})$, explain how this may make diffusion policies more “precise” for fixed T ?

3. Non-normalized variance in DDPM Policies

Suppose for a DDPM-based policy on a multi-dimensional action space, that one action dimension has a non-normalized standard deviation of 0.01. What happens to the signal-to-noise ratio for that dimension across diffusion timesteps, and how would that affect what the model learns?

4. The Copy-Cat Problem

In many applications of behavior cloning, it is common to condition on the history of state-action pairs to provide a policy with additional context beyond the current state. In many real-world regimes, expert actions may be highly autocorrelated (e.g., torque of a joint between two adjacent time steps), which means a model trained on a standard BC objective can achieve low loss by simply copying the action at the previous step. Why might this be a problem when the policy is deployed? Discuss ways to train a policy to avoid this problem.

5. Elite Fraction in CMA-ES

In CMA-ES you refit the search distribution using only the top **elite size** samples out of a population of λ (in Problem 1, the top 10% of 100). Consider the two extremes: keeping only the single best sample, versus keeping the entire population. Describe the failure mode of each extreme in terms of the exploration–exploitation trade-off, and explain the role that the additive covariance noise ϵI plays in preventing the distribution from collapsing prematurely.

6. Gradient-Free vs. Gradient-Based Policy Optimization

CMA-ES optimizes the CartPole policy purely from scalar episode returns, without ever differentiating the reward with respect to the policy parameters, whereas policy-gradient methods estimate $\nabla_{\theta}\mathbb{E}[R(\theta)]$ directly. Give one advantage and one disadvantage of the gradient-free approach. As the number of policy parameters grows from 5 (this problem) to millions (a deep network), which family of methods would you expect to scale better, and why?

7. Why Behavior Cloning Compounds Errors

Suppose a behavior-cloned policy disagrees with the expert with probability at most ϵ on states drawn from the expert's own distribution. A classic analysis shows that the expected number of extra mistakes over a horizon of T steps can grow like $O(\epsilon T^2)$ for BC, but only $O(\epsilon T)$ for DAgger. Explain intuitively (i) why a single early mistake in BC can lead to a quadratic blow-up in error, and (ii) what specifically about DAgger's data-collection procedure (Algorithm 2) eliminates the quadratic term.

8. Action Chunking and the Receding Horizon

The diffusion policy predicts a chunk of k' future actions but only executes `num_actions_to_eval_in_a_row` of them before re-planning (Algorithm 5). Discuss the trade-off, in the `BipedalWalker-v3` setting, between executing *more* versus *fewer* actions per denoising call along three axes: (a) wall-clock inference cost per trajectory, (b) responsiveness to unexpected states, and (c) temporal smoothness of the resulting motion. Relate your answer to the runtimes you measured in Problem 2.3.

References

- [1] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems*, 34:15084–15097, 2021.
- [2] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023.
- [3] Yiming Ding, Carlos Florensa, Pieter Abbeel, and Mariano Phielipp. Goal-conditioned imitation learning. In *Advances in Neural Information Processing Systems*, pages 15324–15335, 2019.
- [4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [5] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International conference on machine learning*, pages 8162–8171. PmLR, 2021.
- [6] Tim Pearce, Tabish Rashid, Anssi Kanervisto, Dave Bignell, Mingfei Sun, Raluca Georgescu, Sergio Valcarcel Macua, Shan Zheng Tan, Ida Momennejad, Katja Hofmann, et al. Imitating human behaviour with diffusion models. *arXiv preprint arXiv:2301.10677*, 2023.
- [7] Stéphane Ross, Geoffrey J Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, volume 1, page 6, 2011.
- [8] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.